DAY 02 STUDY NOTES • PHASE 0: HTML & CSS FOUNDATIONS

Day 2 — HTML Forms, Validation, Radio Buttons & Checkboxes

React + Next.js Roadmap | Connecting Client-Side Data Collection to Java / Spring Boot Architecture

Engineer: Shreyas Kumbhar
Core Stack: Java / Spring Boot to React
Study Focus: Comprehensive Interactive Forms

Executive Study Blueprint & Pedagogical Objective

This document serves as your permanent revision manual for **Day 2**. In this module, we transition from static document markup into interactive data collection: building robust HTML forms, establishing accessible input bindings, enforcing browser validation rules, managing mutual exclusivity with radio buttons, handling multi-select states with checkboxes, and mapping frontend HTTP payload serialization directly to Java Spring Boot REST controllers.

Master Table of Contents

Part I: Form Architecture & Input Controls

- **Chapter 1:** HTML Form Fundamentals & Request Flow
- **Chapter 2:** HTML Attributes Deep Dive: `id`, `name`, `for`, `value`
- **Chapter 3:** Master Guide to HTML Input Types
- **Chapter 4:** Native Browser Form Validation Rules
- **Chapter 5:** HTTP Transport: GET vs POST & Spring Boot Endpoints
- **Chapter 6:** Single Selection Controls: Radio Buttons
- **Chapter 7:** Multi-Selection Controls: Checkboxes

Part II: Project, Verification & Technical Review

- **Chapter 8:** Architectural Comparison: Radio Buttons vs Checkboxes
- **Chapter 9:** Practical Project: Complete Registration Form
- **Chapter 10:** Interactive Browser Validation Practice Run
- **Chapter 11:** Twelve Critical Beginner Mistakes & Fixes
- **Chapter 12:** Java Spring Boot Conceptual Mapping Reference
- **Chapter 13:** 20 Technical Revision Questions & Answer Key
- **Chapter 14:** Day 2 Self-Assessment Checklist & Next Steps

Chapter 1: HTML Form Fundamentals

An **HTML form** is an interactive document region designed to capture user input and package it for transmission to a remote server or client-side processing script. While static HTML elements (`<p>`, `<h1>`, `<section>`) present information, forms enable two-way communication: authentication, search queries, user registration, payments, and data entry.

Core Building Blocks of a Web Form

- `<form>`: The structural wrapper that defines the destination endpoint (`action`) and the HTTP transport method (`method`).
- `<label>`: The human-readable and accessible textual caption associated with an interactive input control.
- `<input>`: The versatile, self-closing void element used to receive various formats of single-line data.
- `<textarea>`: A dedicated, expandable multi-line text input element requiring an explicit closing tag.
- `<button type="submit">`: The interactive trigger that prompts native validation and transmits form data.

Practical Foundation Code Example

```
<form action="/contact" method="post">
  <label for="name">Name:</label>
  <input
    type="text"
    id="name"
    name="name"
    required
  />

  <button type="submit">Submit</button>
</form>
```

Line-by-Line Deconstruction:

- `<form action="/contact" method="post">`: Configures the form to dispatch an HTTP POST request targeting the endpoint path `/contact`.
- `<label for="name">Name:</label>`: Creates an accessible label. The `for="name"` attribute points to the element possessing `id="name"`.
- `<input type="text" id="name" name="name" required />`: Renders a single-line text field. `id="name"` links to the label; `name="name"` defines the payload key; `required` stops submission if left blank.
- `<button type="submit">Submit</button>`: Defines a clickable submit button. Clicking this button triggers the browser's form submission pipeline.

The Form Processing and Submission Pipeline

```
HTML Form Container | |-- <label> (Accessibility label linked to input control) | |-- <input>
```

Spring Boot Request Flow Parallel: In a Spring Boot application, when a user submits this form, the browser issues an `HTTP POST` request containing `name=JohnDoe` in its URL-encoded body. On the server, a controller annotated with `@PostMapping("/contact")` handles the request, binding the parameter directly to a method argument (`@RequestParam("name") String name`) or hydrating a Java Command/DTO object.

Chapter 2: HTML Attributes — id, name, for, value

Understanding the strict boundaries between `id`, `name`, `for`, and `value` is one of the most critical foundational milestones in frontend engineering. Confusing `id` and `name` is the single most common error made by backend engineers transitioning to the browser.

Attribute	Target Element	Core Purpose & Scope	Behavioral Nuance
`id`	Global (Any HTML element)	Uniquely identifies an individual element across the entire DOM document tree.	Must be strictly unique per page. Used by CSS selectors, JavaScript DOM queries, and label targets.
`name`	Form Controls (`input`, `select`, `textarea`)	Identifies the field name within submitted form data sent across the HTTP boundary.	Not required to be unique. Shared across radio buttons to form groups and checkboxes for array binding.
`for`	`<label>` only	Binds the label text programmatically to a specific input control's `id`.	Clicking a label with a matching `for` automatically focuses or toggles the referenced input.
`value`	Form Controls	Represents the data payload associated with the control.	For text inputs, sets default text. For radio/checkbox inputs, defines what string is sent when checked.

Attribute Binding in Practice

```
<label for="email">Email:</label>
<input
  type="email"
  id="email"
  name="email"
  value="shreyas@example.com"
/>
```

The Structural Coupling Relationship

```
label for="email" | matches (Browser accessibility binding & click-to-focus) v input id="emai
```

> **Why `id` and `name` are NEVER Interchangeable:**
> 1. If an input has an `id` but lacks a `name`, the browser will successfully link the label on screen, but *will completely omit the field from the submitted HTTP request*.
> 2. If an input has a `name` but lacks an `id`, the data will reach your Spring Boot server, but *the field will fail accessibility audits* because the screen reader cannot associate the label text with the input.

Chapter 3: Master Guide to HTML Input Types

The `type` attribute instructs the browser how to present the control, what software keyboard to display on mobile devices, and what baseline client-side validation heuristics to apply before dispatching data.

1. Single-Line Text Input (`type="text"`)

Definition & Purpose: Captures standard, unconstrained single-line text strings such as names, cities, or subject titles.

```
<label for="username">Username:</label>
<input type="text" id="username" name="username" minlength="3" maxlength="20" required />
```

Browser Behavior: Accepts any alphanumeric string. Does not apply structural pattern checks unless combined with `minlength` or `pattern`. *Mistake:* Using text inputs for numeric metrics like age or phone numbers.

2. Email Address Input (`type="email"`)

Definition & Purpose: Captures and validates syntactically correct email addresses.

```
<label for="userEmail">Email Address:</label>
<input type="email" id="userEmail" name="userEmail" placeholder="name@domain.com" required />
```

Browser Behavior: Automatically applies a regular-expression check verifying the presence of an `@` character followed by a domain. On mobile platforms, opens a specialized keyboard featuring `@` and `.com` shortcuts.

3. Password Obfuscation Input (`type="password"`)

Definition & Purpose: Captures confidential credentials by masking characters visually with bullet dots or asterisks.

```
<label for="userPassword">Password:</label>
<input type="password" id="userPassword" name="password" minlength="8" required />
```

> **Security Notice for Backend Developers:** A `type="password"` input provides *visual privacy only* against shoulder surfing. It does **not** encrypt, hash, or secure the data across the network. If submitted over plain HTTP, the password travels across the wire in plaintext. Transport-level security (HTTPS/TLS) and backend cryptographic hashing (such as Spring Security's `BCryptPasswordEncoder`) are strictly required.

4. Numeric Metric Input (`type="number"`)

Definition & Purpose: Restricts user input strictly to floating-point or integer numeric quantities.

```
<label for="userAge">Age (Years):</label>
<input type="number" id="userAge" name="age" min="18" max="60" step="1" required />
```

Browser Behavior: Rejects alphabetic characters, renders increment/decrement stepper controls, and summons a numeric dial-pad on mobile viewports. *Mistake:* Using `minlength` instead of `min`.

5. Radio Button Group (`type="radio"`)

Definition & Purpose: Presents a set of mutually exclusive options where a user can select exactly one choice from the set.

6. Checkbox Control (`type="checkbox"`)

Definition & Purpose: Presents independent boolean toggles allowing the selection of zero, one, or multiple items.

Chapter 4: Native Browser Form Validation

Form validation is the programmatic verification that user inputs adhere to expected structural constraints before being accepted. HTML5 provides built-in, declarative client-side validation that executes directly within the browser engine without requiring JavaScript.

Attribute	Compatible Types	Validation Rule & Logic	Valid vs Invalid Scenario
`required`	All inputs, textarea, select	Blocks submission if the input contains empty or whitespace-only data.	Valid: Value present Invalid: Empty field on submit
`placeholder`	text, email, password, textarea	Visual hint; *provides no validation constraints*.	Disappears upon first keystroke.
`minlength`	text, email, password, textarea	Enforces minimum string character count (`string.length >= n`).	For `minlength="3"` : "Shreyas" (7) passes; "Al" (2) fails
`maxlength`	text, email, password, textarea	Prevents entering more than `n` total characters.	For `maxlength="10"` : Browser stops accepting keystrokes at 10.
`min`	number, date, range	Enforces minimum numeric mathematical value (`val >= n`).	For `min="18"` : 25 passes; 17 fails validation
`max`	number, date, range	Enforces maximum numeric mathematical value (`val <= n`).	For `max="60"` : 45 passes; 65 fails validation
`step`	number, range	Specifies the valid numeric increment granularity.	For `step="1"` : 20 passes; 20.5 fails validation

`minlength` vs `min` (Character Count vs Number Value)

`minlength` : Counts the *number of characters in a string*. Used on text, passwords, and textareas. Example: A username with 3 characters ("Sam").

`min` : Evaluates the *mathematical value of a number*. Used on numeric inputs. Example: An applicant's age must be 18 or greater.

`maxlength` vs `max` (String Boundary vs Numeric Ceiling)

`maxlength` : Sets a hard ceiling on *text string length*. The browser physically rejects typing past this limit. Example: A feedback note capped at 200 characters.

`max` : Sets the *maximum allowable numeric magnitude*. Example: Maximum allowable retirement age of 60.

Validation Scenarios from Practice

Evaluated Input	Configured Rule	Browser Verdict	Detailed Engineering Rationale
Name = "Al"	`minlength="3"`	INVALID	String length is 2 characters, failing the required minimum threshold of 3.
Name = "Shreyas"	`minlength="3"`	VALID	String length is 7 characters, comfortably exceeding the minimum of 3.
Age = 17	`min="18"`	INVALID	Numeric value 17 is strictly less than the lower bound of 18.
Age = 25	`min="18"` `max="60"`	VALID	Numeric value 25 sits cleanly within the inclusive range [18, 60].
Message = "Hello"	`minlength="10"`	INVALID	Text contains only 5 characters, triggering browser rejection.
Message = "A message containing 10 characters"	`minlength="10"`	VALID	Text contains 34 characters, satisfying the minimum length constraint.

Client-Side Validation vs Backend Spring Boot Validation:
Browser-side validation provides *immediate user feedback* (saving network trips and improving UX). However, browser validation can be easily bypassed (e.g. via Postman, cURL, or disabling HTML attributes via DevTools). Therefore, backend validation using Java Bean Validation annotations (`@NotNull` , `@Size(min=3)` , `@Min(18)` , `@Email`) alongside `@Valid` in Spring Boot remains strictly mandatory for data integrity and application security.

Chapter 5: GET vs POST & Form Submission

When a form submits, the browser packages all active inputs with a `name` attribute and dispatches an HTTP request. The transmission mechanics are dictated entirely by the `action` and `method` attributes.

Architectural Metric	HTTP GET (`method="get"`)	HTTP POST (`method="post"`)
Data Transmission	Appended directly to URL query string (`/contact?name=Shreyas`).	Encapsulated invisibly within the HTTP Request Body.
Visibility & History	Visible in address bar, stored in browser history and server access logs.	Hidden from address bar; never retained in browser navigation history.
Payload Limits	Severely restricted by browser URL length limits (~2,048 characters).	Virtually limitless; supports large text payloads and file uploads.
Idempotency & Safety	Idempotent & Safe. Re-executing queries does not mutate server state.	Non-idempotent. Used for state-mutating actions (create, update, delete).
Primary Use Cases	Search bars, filter criteria, public pagination queries.	User registration, login credentials, payment checkouts.

Connecting HTML Form Action to Spring Boot REST Endpoints

```
<!-- Frontend Client Form -->
<form action="/contact" method="post">
  <input type="text" id="name" name="name" required />
  <button type="submit">Submit</button>
</form>
```

```
// Backend Spring Boot Controller (Conceptual Mapping)
@RestController
public class ContactController {

    @PostMapping("/contact")
    public String contact() {
        return "Contact form received";
    }
}
```

Conceptual Endpoint Coupling: The form attribute `action="/contact"` specifies the exact URI route mapped by the Spring Boot annotation `@PostMapping("/contact")` . The attribute `method="post"` guarantees that the browser issues an HTTP POST request matching the controller's verb. *Note: This basic Java method confirms route connectivity; reading submitted fields requires binding arguments like* `@RequestParam("name") String name` .

Chapter 6: Single Selection Controls: Radio Buttons

Radio buttons (`<input type="radio">`) are used when a user must choose exactly one option from a predefined list of mutually exclusive choices. The defining characteristic of radio buttons is their group behavior.

```
<section>
  <h2>Select Gender:</h2>

  <label for="male">Male</label>
  <input
    type="radio"
    id="male"
    name="gender"
    value="Male"
    required
  />

  <label for="female">Female</label>
  <input
    type="radio"
    id="female"
    name="gender"
    value="Female"
  />

  <label for="other">Other</label>
  <input
    type="radio"
    id="other"
    name="gender"
    value="Other"
  />
</section>
```

Structural Option Hierarchy

```
        Gender (Logical Group Concept) | |-- Male (id="male", value="Male") |-- Female (id="female",
```

- **Shared Name:** All three radio buttons share `name="gender"`. This shared name is what informs the browser that they belong to the same group.
- **Unique IDs:** Each individual radio input must have a distinct `id` so its corresponding `<label for="...">` connects to it correctly.
- **Mandatory Value:** If `value` is omitted, the browser submits the useless default string `"on"` instead of the selected choice.
- **Group-Level Required:** Adding `required` to a single radio button makes selecting at least one option mandatory for the entire group.

Submitted HTTP Payload Example:

```
gender=Female
```

Selecting "Female" automatically unchecks "Male" or "Other". Only the single checked value is included in the submission payload.

Chapter 7: Multi-Selection Controls: Checkboxes

Checkboxes (`<input type="checkbox">`) are used when items represent independent toggles. Users can select zero, one, or several options from a collection.

```
<section>
  <h2>Select Skills:</h2>

  <label for="java">Java</label>
  <input
    type="checkbox"
    id="java"
    name="skills"
    value="Java"
  />

  <label for="html">HTML</label>
  <input
    type="checkbox"
    id="html"
    name="skills"
    value="HTML"
  />

  <label for="css">CSS</label>
  <input
    type="checkbox"
    id="css"
    name="skills"
    value="CSS"
  />
</section>
```

Structural Option Hierarchy

```
        Skills (Multi-Value Collection) | |-- Java (id="java", value="Java") |-- HTML (id="html", va
```

- **Multiple Selections Allowed:** Toggling one box has zero effect on the others. Users can select any combination (e.g., Java only, Java + HTML, or all three).
- **Unchecked Omission Rule:** Unchecked checkboxes are *completely omitted* from the submitted payload. If CSS is unchecked, no `skills=CSS` parameter is sent.
- **Array Binding:** When multiple boxes share the same `name="skills"`, browsers send repeated key-value pairs, which Spring Boot automatically converts into a `List<String>`.

Submitted HTTP Payload Example (Java & HTML Checked):

```
skills=Java&skills=HTML
```

Chapter 8: Architectural Comparison: Radio Buttons vs Checkboxes

Both radio buttons and checkboxes allow users to select from preset options, but their selection mechanics and data representations serve fundamentally different purposes:

Feature Metric	Radio Buttons (`type="radio"`)	Checkboxes (`type="checkbox"`)
Selection Rule	**Strictly Mutually Exclusive:** Selecting one automatically deselects all others in the group.	**Independent Multi-Select:** Each box toggles independently of all others.
Selection Cardinality	Exactly 1 option per group (when a selection is made).	0, 1, or many options.
Common Real-World Uses	Gender selection, billing payment method (Card vs PayPal), shipping speed tier.	Technical skillsets, notification preferences, agreeing to Terms & Conditions.
Role of Shared `name`	**Enforces Mutual Exclusivity:** Ties the radio buttons into a unified single-choice set.	**Groups Related Values:** Groups items under a shared key, binding to an array on the server.
Role of `value`	Defines the single string payload submitted for the entire group.	Defines the individual item string sent for each checked box.
Spring Boot Representation	Single scalar value: `String gender` or Java `Enum`.	Collection or Boolean: `List<String> skills` or `boolean agreed`.

Quick Reference Rule:
• **Radio Buttons:** *Choose exactly one* (mutually exclusive alternatives).
• **Checkboxes:** *Choose any number* (independent, non-exclusive toggles).

Chapter 9: Practical Project: Complete Registration Form

The following complete, valid HTML document was built during Day 2 practice. It brings together all the core concepts learned: structural containers, form fields, validation constraints, radio groups, multi-select checkboxes, and multi-line text entry.

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta
      name="viewport"
      content="width=device-width, initial-scale=1.0"
    />
    <title>Registration Form</title>
  </head>

  <body>
    <header>Registration Form</header>

    <form action="/contact" method="post">
      <label for="name">Name:</label>

      <input
        type="text"
        id="name"
        name="name"
        minlength="3"
        placeholder="enter your name"
        required
      />

      <label for="email">Email:</label>

      <input
        type="email"
        id="email"
        name="email"
        placeholder="enter your email"
        required
      />

      <label for="password">Password:</label>

      <input
        type="password"
        id="password"
        name="password"
        placeholder="enter password"
        minlength="8"
        required
      />

      <label for="age">Age:</label>

      <input
        type="number"
        id="age"
        name="age"
        placeholder="enter your age"
        min="18"
        max="60"
        step="1"
        required
```

Parent-Child DOM Architecture Tree

```
html | |-- head | |-- meta (charset) | |-- meta (viewport) | |-- title | |-- body | |-- heade
```

Chapter 10: Browser Validation Practice Run

During the Day 2 practical exercise, we evaluated how the browser engine handles a partially valid registration form submission:

Submitted Test Values:

- **Name:** "Al"
- **Email:** "shreyas@gmail.com"
- **Age:** 17
- **Message:** "Hello"

Evaluated Field	Configured Rule	Validation Result	Detailed Browser Rejection Reason
Name	`minlength="3"`	INVALID	"Al" contains only 2 characters; fails minimum threshold of 3.
Email	`type="email"`	VALID	"shreyas@gmail.com" adheres to standard email format.
Age	`min="18"`	INVALID	17 is below the required minimum numeric boundary of 18.
Message	`minlength="10"`	INVALID	"Hello" contains only 5 characters; fails minimum of 10.
Form Submission	Overall Form Check	BLOCKED	**The browser completely halts form submission.**

> **The "All-or-Nothing" Form Submission Rule:** A valid email address does not allow the form to submit. Browser form validation operates on a strict logical conjunction (`AND` condition): *Every single required constraint across all fields must pass simultaneously*. If even one field fails validation, the browser cancels submission, focuses the first invalid element, and displays a localized validation tooltip to the user.

Chapter 11: Twelve Critical Beginner Mistakes & Fixes

Anti-Pattern / Common Mistake	Erroneous Syntax	Correct Syntax & Engineering Solution
1. `min` instead of `minlength` on text	`<input type="text" min="3">`	`<input type="text" minlength="3">` `min` has no effect on text strings; `minlength` counts characters.
2. `minlength` instead of `min` on numbers	`<input type="number" minlength="18">`	`<input type="number" min="18">` `minlength="18"` would require typing an 18-digit number!
3. Different names on radio group	`<input type="radio" name="g1">` `<input type="radio" name="g2">`	`<input type="radio" name="gender">` `<input type="radio" name="gender">` Radio buttons only become mutually exclusive when sharing the same `name`.
4. Reusing the same ID across options	`<input type="radio" id="opt">` `<input type="radio" id="opt">`	`<input type="radio" id="male">` `<input type="radio" id="female">` Duplicate IDs break accessibility and cause labels to target the wrong input.
5. Missing label-input link	`<label>Name:</label>` `<input id="name">`	`<label for="name">Name:</label>` `<input id="name">` Missing `for` breaks accessibility and disables click-to-focus behavior.

Anti-Pattern / Common Mistake	Erroneous Syntax	Correct Syntax & Engineering Solution
6. Confusing `id` and `name`	`<input id="age">` (missing `name`)	`<input id="age" name="age">` Form inputs without a `name` are completely omitted from the HTTP payload.
7. Forgetting value on choices	`<input type="radio" name="g">`	`<input type="radio" name="g" value="M">` Without `value`, the browser submits the useless default string `g=on`.
8. Using text type for numeric data	`<input type="text" name="age">`	`<input type="number" name="age">` `type="number"` enforces numeric validation and opens the dial-pad on mobile.
9. Expecting required to check length	`<input type="text" required>`	`<input type="text" minlength="3" required>` `required` only checks that the field isn't empty; it doesn't check character length.
10. Relying only on HTML validation	Omitting backend checks	Always combine HTML validation with Spring Boot Bean Validation (`@Valid`).
11. Expecting unchecked boxes to submit	Expecting `skills=false`	Understand that unchecked checkboxes are omitted from form payloads entirely.
12. Assuming action builds backend	`<form action="/api">`	The backend route must still be implemented in a controller (`@PostMapping`).

Chapter 12: Java and Spring Boot Conceptual Mapping

As a backend developer transitioning to frontend engineering, mapping browser-side constructs to familiar Spring Boot paradigms establishes an intuitive mental model:

HTML Form Construct	Java / Spring Boot Concept	Spring Boot Code Equivalent
Form field `name`	Request parameter key or DTO field name	`@RequestParam("name") String name` or field in `RegistrationDto`
Input `value`	Submitted argument or bound entity property	The runtime value populated into the DTO field
Form `action="/contact"`	REST Controller mapping path	`@PostMapping("/contact")` or `@RequestMapping`
Form `method="post"`	HTTP POST request verb annotation	`@PostMapping` (requires request body reading)
Browser Validation (`minlength`, `required`)	Client-side preliminary validation (UX feedback)	Precursor to Spring Bean Validation (`@Valid`)
Backend Validation	Server-side authoritative security rules	`@NotNull`, `@Size(min=3)`, `@Min(18)`, `@Email`
Radio Button Selected Value	Single scalar value or Java Enum	`Gender gender` (where `Gender` is an Enum)
Checkbox Shared Values	Multi-valued Collection or List	`@RequestParam("skills") List<String> skills`

> **Separation of Concerns:** HTML is solely responsible for collecting user input, providing immediate visual feedback, and packaging data into an HTTP request. Spring Boot is solely responsible for receiving the request, enforcing business rules, validating data integrity, and persisting state to the database.

Chapter 13: Day 2 Revision Questions (20 Questions)

Test your understanding of Day 2 concepts. Detailed answers are provided in the answer key below.

Part 1: Conceptual Architecture Questions (1–10)

1. What is the core functional difference between the `id` and `name` attributes on an input element?
2. Why must a `<label>` element's `for` attribute match an `<input>`'s `id` rather than its `name`?
3. What happens to a form control if its `name` attribute is completely omitted during form submission?
4. What is the fundamental difference between `minlength` and `min`?
5. What is the fundamental difference between `maxlength` and `max`?
6. How does a browser group radio buttons together to enforce mutual exclusivity?
7. Why does a set of related checkboxes allow multiple selections while a radio group does not?
8. What does the boolean attribute `required` enforce, and what limitation does it have regarding string length?
9. Why is HTTP POST preferred over HTTP GET for submitting user registration and password data?
10. Why is client-side HTML validation alone insufficient for real-world enterprise applications?

Part 2: Code Reading & Analysis Questions (11–15)

11. Analyze this snippet: `<input type="password" id="pass" name="pass" minlength="8">`. Does this encrypt the password over the wire?
12. Analyze this snippet: `<input type="radio" id="r1" name="theme1" value="dark">` and `<input type="radio" id="r2" name="theme2" value="light">`. Can both be checked simultaneously? Why?
13. Analyze this snippet: `<input type="checkbox" id="c1" name="alert">` without a `value` attribute. What data is submitted if checked?
14. What is submitted across the HTTP boundary if all checkboxes in a group are left unchecked?
15. Analyze this snippet: `<textarea id="bio" name="bio" rows="4">Text</textarea>`. Why does `<textarea>` use closing tags instead of a `value` attribute?

Part 3: Practical Validation Scenarios (16–20)

16. A text field has `minlength="5"`. The user enters "Java" and clicks submit. What does the browser do?
17. An age field has `type="number" min="18" max="60"`. The user enters 16. What occurs on submission?
18. A form has 5 fields. 4 fields are completely valid, but 1 field fails its `minlength` constraint. Will the form submit?
19. A radio button group has 3 options. The first option has `required`. Does the user have to select the first option specifically?
20. If an `<input type="number">` has `step="1"`, will the browser accept 21.5 as valid input?

Chapter 13 Answer Key

Answers: Questions 1–10

1. id vs name: `id` uniquely identifies an element within the DOM (used for CSS, JS, and labels). `name` identifies the parameter key sent in the HTTP request payload.
2. label for ↔ input id: Assistive software and browser click-to-focus events rely on the unique DOM identifier (`id`) to locate the exact node.
3. Omitted name: The browser completely omits the input's data from the submitted form payload.
4. minlength vs min: `minlength` checks character count on text strings; `min` checks numeric value magnitude on numbers/dates.
5. maxlength vs max: `maxlength` limits maximum string character length; `max` defines maximum numeric boundary.
6. Radio grouping: By assigning the exact same `name` attribute across all radio buttons in that set.
7. Checkbox multi-select: Checkboxes represent independent boolean toggles; selecting one does not uncheck others.
8. required rule: It requires the field to have a value, but doesn't check character length (a single character satisfies `required`).
9. POST over GET: POST transmits data in the request body rather than exposing credentials in the URL query string and browser history.
10. Client validation limits: Browser validation can be bypassed easily using tools like Postman, cURL, or disabling HTML attributes via DevTools.

Answers: Questions 11–20

11. Password encryption: No. `type="password"` only masks characters visually on screen; transport encryption requires HTTPS.
12. Multiple radio checks: Yes, both can be checked because they have different names (`theme1` vs `theme2`), making them separate groups.
13. Checkbox without value: The browser submits the default string value `alert=on` .
14. Unchecked checkboxes: Nothing is submitted; unchecked checkboxes are completely omitted from the HTTP payload.
15. Textarea closing tag: `<textarea>` supports multi-line text and line breaks, which cannot be represented inside an attribute value string.
16. "Java" with minlength="5": The browser stops submission, highlights the input, and displays a validation error message.
17. Age 16 with min="18": The browser blocks submission and alerts the user that the value must be greater than or equal to 18.
18. 4 of 5 fields valid: No. All required validation rules must pass simultaneously before submission can proceed.
19. required on first radio: No. It simply requires that *at least one* option in the group is selected.
20. step="1" with 21.5: Rejected as invalid because 21.5 is not an integer multiple of the step value (1).

Chapter 14: Day 2 Summary & Revision Checklist

Day 2 Self-Assessment Checklist (Review each item before proceeding to Day 3)

- [ ] Understand HTML form structure (`<form>` , `<label>` , `<input>` , `<textarea>` , `<button>`)
- [ ] Understand the critical distinction between client `id` and server `name`
- [ ] Can link `<label for="...">` to `<input id="...">` correctly
- [ ] Understand the purpose of `value` across text, radio, and checkbox inputs
- [ ] Understand standard input types (`text` , `email` , `password` , `number`)
- [ ] Understand `required` validation and `placeholder` helper text
- [ ] Understand string length constraints (`minlength` vs `maxlength`)
- [ ] Understand numeric boundary constraints (`min` , `max` , and `step`)
- [ ] Can group radio buttons using a shared `name` for mutual exclusivity
- [ ] Can configure multi-select checkboxes and handle submitted array values
- [ ] Understand the architectural differences between HTTP GET and POST
- [ ] Understand how browser validation works and why backend validation is also required
- [ ] Understand how HTML form fields map to Spring Boot controllers (`@PostMapping` , DTOs)
- [ ] Completed building the full registration form practice project

Next Learning Step

According to your React + Next.js roadmap, you will continue reinforcing your **HTML form mechanics and validation rules** through hands-on practice. Build additional form variations (e.g. login screens, product order forms, search filters) until structuring accessible, validated forms feels natural. Once comfortable with HTML fundamentals, you will move forward to CSS styling (box model, flexbox, and responsive design) to transform these functional forms into polished user interfaces.